

Получение $\mathbf{M}(\mathbf{q})$ и $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$ из модели Flexia для планирования оптимального по быстродействию движения

Обозначения

$\mathbf{p} \in \mathbb{R}^{3n_b}$ — абсолютные координаты положений тел, $\mathbf{v} = \dot{\mathbf{p}}$; $\lambda \in \mathbb{R}^m$ — множители Лагранжа; $\mathbf{q} \in \mathbb{R}^d$ — суставные координаты, $d = 3n_b - \text{rank } \mathbf{C}$; $\Phi(\mathbf{p}) \in \mathbb{R}^m$ — связи; $\mathbf{C} = \partial\Phi/\partial\mathbf{p} \in \mathbb{R}^{m \times 3n_b}$; $\mathbf{M}_a \in \mathbb{R}^{3n_b \times 3n_b}$ — матрица масс в абсолютных координатах.

Полный набор координат обозначен $\mathbf{p}$, минимальный — $\mathbf{q}$; это разные векторы. Знак при λ принят как в коде (вклад реакций в правой части).

1. Задача

Планировщик требует модель в явной форме

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}), \quad \tau_{\min} \leq \tau \leq \tau_{\max}, \quad (1)$$

плюс путь $\mathbf{q}(s)$, $s \in [0, 1]$, дважды дифференцируемый. Такая форма нужна потому, что после подстановки $\dot{\mathbf{q}} = \mathbf{q}'\dot{s}$ ограничение на моменты становится линейным по $(\ddot{s}, \dot{s}^2)$ и задача выпукла.

Flexia даёт систему в абсолютных координатах

$$\mathbf{M}_a \dot{\mathbf{v}} = \mathbf{Q}(t, \mathbf{p}, \mathbf{v}) + \mathbf{C}^\top \lambda, \quad \Phi(\mathbf{p}) = \mathbf{0}, \quad (2)$$

где $\mathbf{M}_a$ постоянна и вырождена, размерность 96 против $d = 4$, управление не выделено. Переход от (2) к (1) — редукция к минимальным координатам, изложена ниже.

Сборка модели. Для вычисления $\mathbf{M}$ и $\mathbf{h}$ модель собирается без PositionMotor2D и PositionLinearActuator2D: в текущей реализации привод — кинематическая связь (actuators.jl:18,44), а не источник обобщённой силы. Четыре привода в parmec.jl дали бы $d = 0$. Приводные углы должны оставаться свободными координатами, моменты — входить в правую часть как обобщённые силы.

2. Редукция

Скорости. Дифференцируя $\Phi(\mathbf{p}) = \mathbf{0}$, получаем $\mathbf{C}\mathbf{v} = \mathbf{0}$. Разделим индексы на независимые $\mathcal{I}$ (d штук, угловые координаты кривошипов) и зависимые $\mathcal{D}$ (m штук). Тогда $\mathbf{C}_{\mathcal{I}}\mathbf{v}_{\mathcal{I}} + \mathbf{C}_{\mathcal{D}}\mathbf{v}_{\mathcal{D}} = \mathbf{0}$, откуда при $\mathbf{q} := \mathbf{p}_{\mathcal{I}}$

$$\mathbf{v} = \mathbf{R}\dot{\mathbf{q}}, \quad \mathbf{R}_{\mathcal{I},:} = \mathbf{I}_d, \quad \mathbf{R}_{\mathcal{D},:} = -\mathbf{C}_{\mathcal{D}}^{-1}\mathbf{C}_{\mathcal{I}}, \quad \mathbf{C}\mathbf{R} = \mathbf{0}. \quad (3)$$

Столбцы $\mathbf{R}$ — базис $\ker \mathbf{C}$, то есть касательное пространство к многообразию $\Phi = \mathbf{0}$.

Ускорения. Второе дифференцирование даёт $\mathbf{C}\dot{\mathbf{v}} = \boldsymbol{\gamma}$, $\boldsymbol{\gamma} = -\dot{\mathbf{C}}\mathbf{v}$, откуда

$$\dot{\mathbf{v}} = \mathbf{R}\ddot{\mathbf{q}} + \mathbf{s}, \quad \mathbf{s}_{\mathcal{I}} = \mathbf{0}, \quad \mathbf{s}_{\mathcal{D}} = \mathbf{C}_{\mathcal{D}}^{-1}\boldsymbol{\gamma}. \quad (4)$$

Сопоставление с дифференцированием (3) даёт $\mathbf{s} = \mathbf{R}\ddot{\mathbf{q}}$; явное дифференцирование $\mathbf{R}$ не требуется. Величина $\boldsymbol{\gamma}$ есть производная по направлению и берётся автодифференцированием:

$$\boldsymbol{\gamma} = - \left. \frac{d}{d\varepsilon} \right|_{\varepsilon=0} \mathbf{C}(\mathbf{p} + \varepsilon\mathbf{v})\mathbf{v}. \quad (5)$$

Проекция. Подставляя (4) в (2) с добавлением моментов приводов и умножая слева на $\mathbf{R}^\top$, получаем $\mathbf{R}^\top \mathbf{C}^\top = (\mathbf{C}\mathbf{R})^\top = \mathbf{0}$ (реакции идеальных связей ортогональны допустимым движениям), и множители исчезают:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{R}^\top \mathbf{B}\boldsymbol{\tau}, \quad \mathbf{M} = \mathbf{R}^\top \mathbf{M}_a \mathbf{R}, \quad \mathbf{h} = \mathbf{R}^\top \mathbf{M}_a \mathbf{s} - \mathbf{R}^\top \mathbf{Q}. \quad (6)$$

Матрица $\mathbf{M}$ симметрична и положительно определена при $\text{rank } \mathbf{R} = d$; это же следует из $\frac{1}{2}\mathbf{v}^\top \mathbf{M}_a \mathbf{v} = \frac{1}{2}\dot{\mathbf{q}}^\top \mathbf{M} \dot{\mathbf{q}}$.

Столбец $\mathbf{B}$ для момента τ_k содержит ± 1 в угловых строках двух тел шарнира. Если привод соединяет закреплённое основание с телом, угол которого выбран независимой координатой, то $\mathbf{R}^\top \mathbf{B} = \mathbf{I}_d$ и $\boldsymbol{\tau} = \mathbf{M}\ddot{\mathbf{q}} + \mathbf{h}$; иначе произведение считается явно.

3. Извлечение исходных данных из Flexia

Строки множителей в правой части содержат в точности Φ (присваивание, а не накопление: `joints.jl:68–70, 157–161, 391–395`), поэтому $\mathbf{C}$ доступна как блок якобиана (`system.jl:117`). Вычитание уставок (`system.jl:112`) на производную не влияет.

```
lm_rows = collect((last_body_dof(sys)+1):last_lm_dof(sys))
pos_cols = vcat([get_body_position_dofs(sys, b) for b in sys.bodies]...)
vel_cols = vcat([get_body_velocity_dofs(sys, b) for b in sys.bodies]...)

C = sys.jacobian(state)[lm_rows, pos_cols] # m x 3n_b
Ma = sys.mass[vel_cols, vel_cols] # 3n_b x 3n_b, постоянная

st = copy(state); st[lm_rows] .= 0.0 # вклады связей линейны по L
Q = sys.rhs(st)[vel_cols] # чистые внешние силы
```

Порядок индексов в `pos_cols` задаёт нумерацию координат для всех матриц и должен совпадать с порядком `vel_cols` и с разбиением $\mathcal{I}/\mathcal{D}$. Несогласованность даёт результат, сохраняющий симметричность и положительную определённость $\mathbf{M}$, то есть не выявляемый стандартными проверками.

Прямой путь через `sys.kinematic_constraints!` невозможен: сигнатуры `Vector{Float64}` в `compute_kinematic_residual!` (`joints.jl:72, 172, 348`) не пропускают дуальные числа. Замена на `AbstractVector{<:Real}` сделала бы обходной путь ненужным.

Целесообразно оформить всё перечисленное отдельным модулем `src/reduction.jl` со структурой, фиксирующей списки индексов, и функциями `constraint_jacobian`, `nullspace_basis`, `mass_matrix`, `bias_vector`, `assemble_configuration!`. Дополнительно нужен тип `TorqueActuator2D <: AbstractForce2D` — привод как источник момента (по устройству аналогичен `TorsionalSpring`, `forces.jl:25–53`, `number_of_dofs` равен нулю). Сейчас моменты приходится подавать через поле `forces` тел.

4. Путь и форма для планировщика

Путь $\mathbf{q}(s)$ задаётся либо напрямую сплайном по узлам в суставных координатах, либо через обратную задачу: для каждого s_k решается $[\Phi(\mathbf{p}); \mathbf{g}(\mathbf{p}) - \mathbf{g}_{\text{зад}}(s_k)] = \mathbf{0}$ методом Ньютона со стартом от предыдущего узла, из решения берутся компоненты $\mathcal{I}$. Второй способ требует функции сборки конфигурации; `static_solver!` для этого не годится.

Подстановка $\dot{\mathbf{q}} = \mathbf{q}'\dot{s}$, $\ddot{\mathbf{q}} = \mathbf{q}'\ddot{s} + \mathbf{q}''\dot{s}^2$ в (6) даёт

$$\tau(s) = \mathbf{a}(s)\ddot{s} + \mathbf{b}(s)\dot{s}^2 + \mathbf{c}(s), \quad (7)$$

$$\mathbf{a} = \mathbf{M}(\mathbf{q})\mathbf{q}', \quad \mathbf{c} = \mathbf{h}(\mathbf{q}, \mathbf{0}), \quad \mathbf{b} = \mathbf{M}(\mathbf{q})\mathbf{q}'' + \mathbf{h}(\mathbf{q}, \mathbf{q}') - \mathbf{c}. \quad (8)$$

Использовано то, что $\mathbf{h}$ есть сумма однородного второй степени по скорости слагаемого и позиционного. Матрицу Кориолиса отдельно вычислять не нужно: все коэффициенты получаются тремя вызовами $\mathbf{M}$ и $\mathbf{h}$.

Разложение верно, только если $\mathbf{Q}$ не зависит от скоростей. Демпфирование (поле `damping` в `TorsionalSpring`) добавляет член, линейный по $\dot{s}$, что нарушает выпуклость. В `parmech.jl` диссипации нет.